Arquitectura ResNet18 para clasificación de flores

Implementación y verificación de una red residual con transferencia de aprendizaje sobre Oxford Flowers102. Se compararon seis configuraciones y se seleccionó E5_LAYER4 mediante validación. El modelo final registrado en GPU obtuvo 91.9608% en validación y 88.8600% Top-1 en prueba.

1 Arquitectura y adaptación

Se cargó ResNet18 con ResNet18_Weights.DEFAULT y se sustituyó fc por Linear(512, 102). La imagen RGB se redimensiona a 256, se recorta al centro a 224 × 224 y se normaliza con media [0.485, 0.456, 0.406] y desviación [0.229, 0.224, 0.225]. Los bloques residuales combinan la transformación convolucional y una conexión de identidad; downsample ajusta dimensiones cuando cambian canales o resolución.

Componente	Salida sin lote	Parámetros
Entrada	3 × 224 × 224	—
conv1	64 × 112 × 112	9,408
bn1	64 × 112 × 112	128
maxpool	64 × 56 × 56	0
layer1	64 × 56 × 56	147,968
layer2	128 × 28 × 28	525,568
layer3	256 × 14 × 14	2,099,712
layer4	512 × 7 × 7	8,393,728
avgpool	512 × 1 × 1	0
fc	102 logits	52,326

ReLU conserva la forma de los tensores. El pooling adaptativo produce 512 características y el aplanado las entrega a fc. Los forward hooks verificaron las dimensiones sobre una instancia del modelo.

El total es 11,228,838 parámetros. B0 entrena solo fc: 52,326 entrenables y 11,176,512 congelados. E5 entrena layer4 y fc: 8,446,054 entrenables y 2,782,784 congelados. El optimizador se reconstruye con los parámetros habilitados.

2 Metodología y experimentos

Se utilizaron las particiones oficiales: entrenamiento 1,020 imágenes, validación 1,020 y prueba 6,149, con 102 clases en cada una. Entrenamiento y validación contienen 10 ejemplos por clase; prueba tiene entre 20 y 238. Se verificaron etiquetas, formas y ausencia de rutas compartidas entre particiones.

Cada configuración inició desde pesos preentrenados y semilla declarada. Se utilizó CrossEntropyLoss sobre logits y Adam. Cada lote ejecuta zero_grad, forward, cálculo de pérdida, backward y step. En validación se emplean eval y no_grad. Las BatchNorm de capas congeladas permanecen en evaluación; las de layer4 permanecen en entrenamiento cuando se ajusta esa etapa.

ID	Cambio frente a B0	Validación	Tiempo GPU
B0	Semilla 42; lote 32; LR 0.001; 5 épocas	75.6863%	65.29 s
E1_SEMILLA	Semilla 7	76.3725%	62.91 s
E2_LOTE	Lote 64	73.4314%	60.63 s
E3_LR	LR 0.0001	18.3333%	63.62 s
E4_EPOCAS	10 épocas	83.4314%	128.24 s
E5_LAYER4	Entrenar layer4 y fc	91.9608%	63.84 s

La semilla alternativa aumentó validación 0.6863 puntos porcentuales; no basta una repetición para concluir estabilidad. El lote 64 redujo tiempo, pero disminuyó exactitud 2.2549 puntos. Reducir LR produjo menor avance con cinco épocas. Ampliar a diez épocas mejoró 7.7451 puntos; habilitar layer4 mejoró 16.2745 puntos.

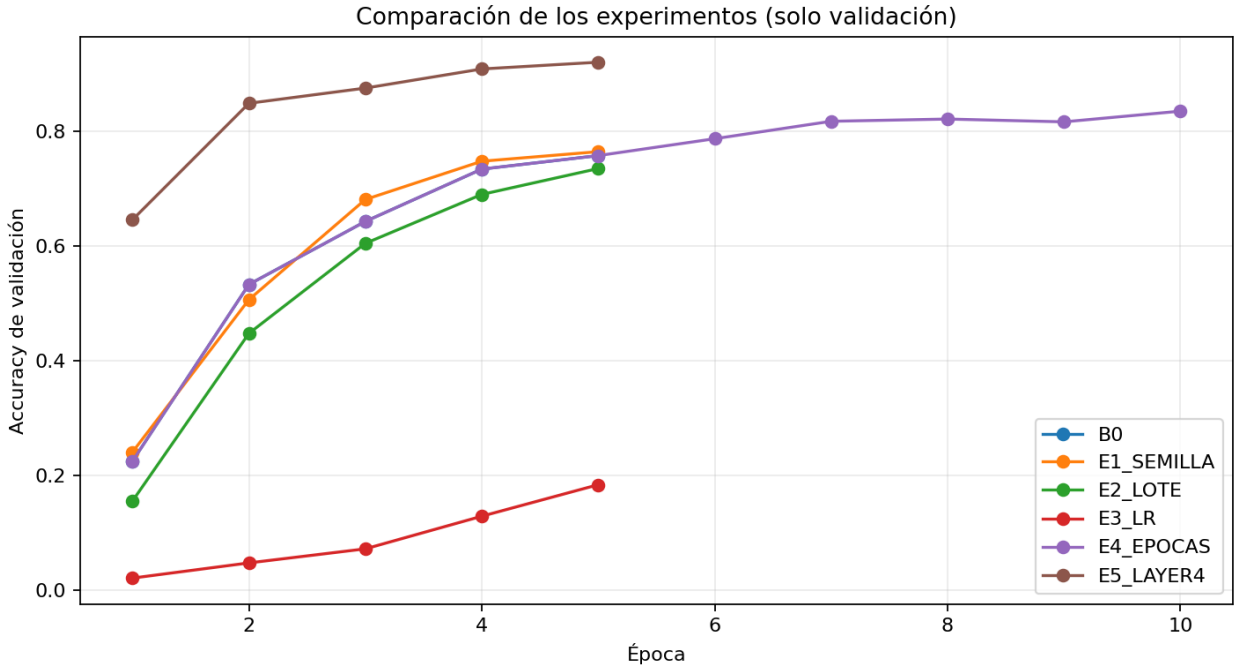
Selección y evaluación

Se conservó el checkpoint de mayor exactitud de validación, con menor pérdida como desempate por época. Entre experimentos, los desempates previstos fueron pérdida, número de parámetros entrenables e ID. E5_LAYER4 fue seleccionado en la época 5 antes de evaluar prueba en esa ejecución.

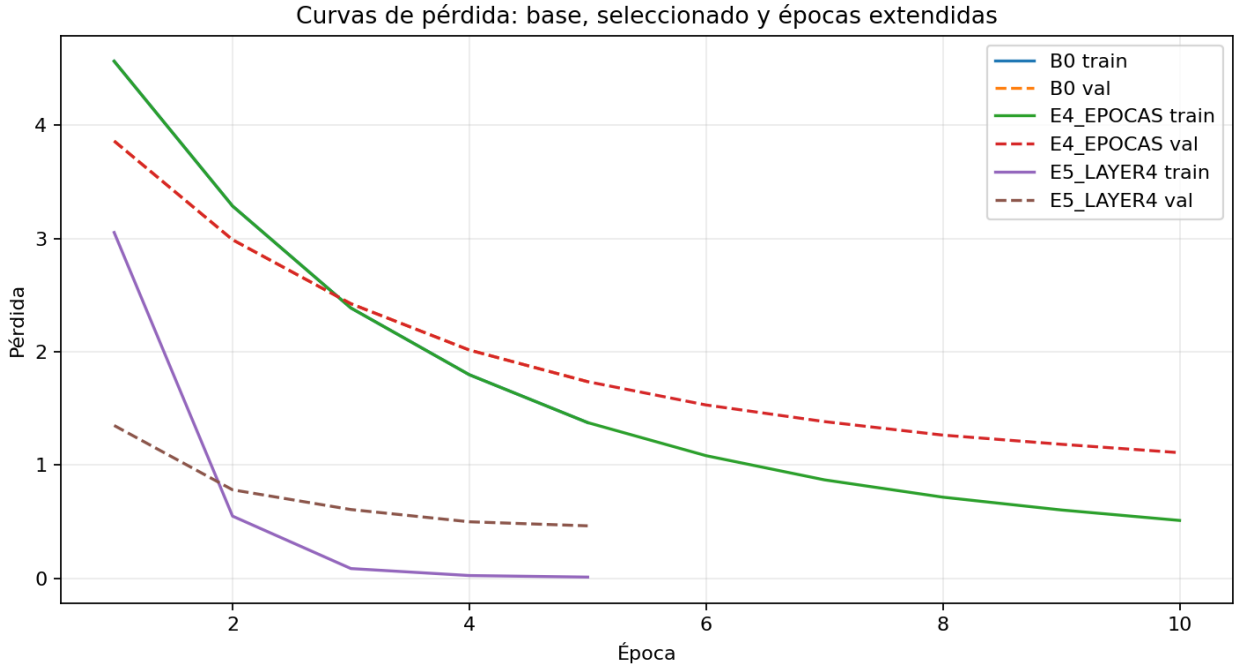
Prueba GPU	Resultado
Imágenes	6,149
Pérdida	0.5491966
Top-1	88.8600%: 5,464 aciertos y 685 errores
Top-3	94.9748%

Top-3 supera Top-1 en 6.1148 puntos porcentuales. El reporte por clase y la matriz permiten examinar errores que una cifra global de accuracy no describe.

3 Curvas y hallazgos

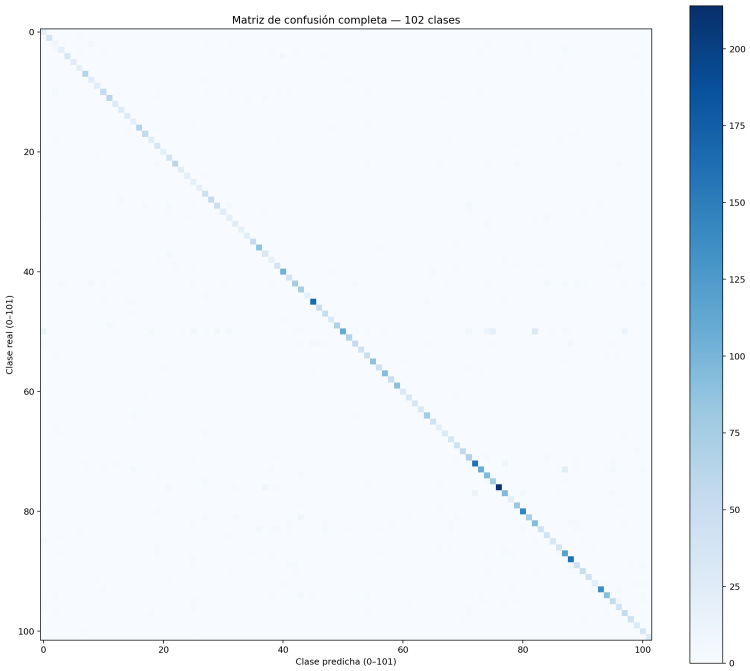


La mayor exactitud de validación corresponde a E5. E3 presenta menor avance durante las cinco épocas. E4 prolonga la mejora observada en B0 hasta diez épocas. Los resultados describen las configuraciones registradas; no establecen un óptimo universal.

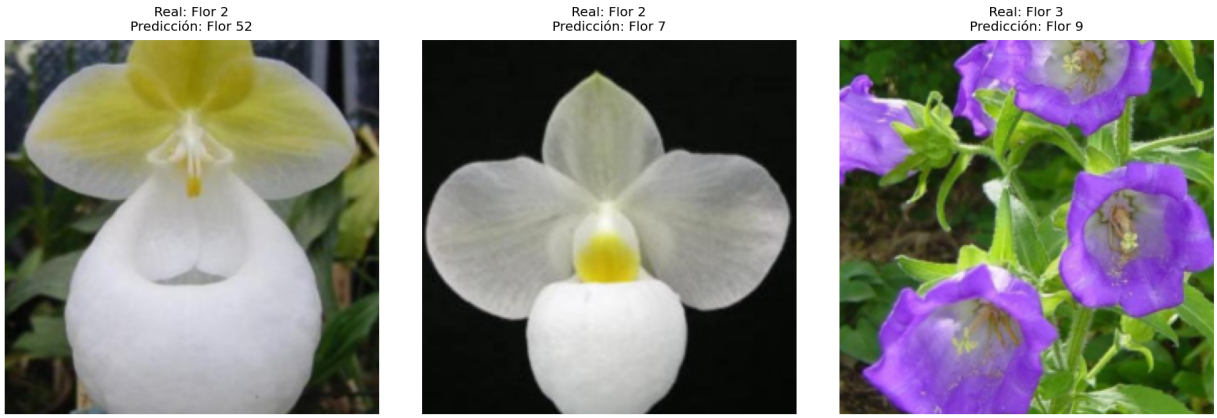


B0 redujo pérdida de entrenamiento de 4.5653 a 1.3747 y de validación de 3.8607 a 1.7359. E4 terminó con pérdida de validación 1.1090 y E5 con 0.4631. En E5 existe separación entre ambas pérdidas, pero validación mejora durante las cinco épocas. No se observa un deterioro sostenido de validación que justifique afirmar sobreajuste en ese intervalo.

4 Errores y comportamiento por clase



La matriz registra las 102 clases, con índices 0–101. Las confusiones más frecuentes en la ejecución GPU fueron Flor 51 → Flor 83 (31 casos) y Flor 74 → Flor 88 (23 casos). El CSV por clase conserva precisión, recall, F1 y soporte. Las etiquetas Flor 1–102 identifican categorías del dataset, no nombres botánicos.



En el primer error (Flor 2 → Flor 52) se observa una flor blanca con región superior amarilla y un pétalo inferior grande; en el segundo (Flor 2 → Flor 7), pétalos blancos sobre fondo oscuro y un centro amarillo; en el tercero (Flor 3 → Flor 9), varias corolas moradas en forma de campana y vegetación alrededor. Los dos primeros comparten etiqueta real, pero difieren en encuadre, fondo y distribución del amarillo. El tercero contiene varias flores, a diferencia de los primeros planos anteriores. Estas diferencias son observables en las imágenes; no demuestran por sí solas la causa de las predicciones incorrectas.

En los cinco ejemplos impresos, Flor 1 fue etiqueta real y primera predicción; también estuvo en Top-3. Esta muestra limitada no representa el conjunto de categorías. Las probabilidades softmax no demuestran calibración.

5 Exportación y reproducibilidad

Se exportaron state_dict y metadatos del modelo final GPU. El checkpoint distribuido tiene 44,998,667 bytes y declara CUDA, E5_LAYER4, 102 clases y la configuración utilizada. Su identificación SHA-256 se conserva en docs/VERIFICACION_GPU.md y en el manifiesto del paquete.

El notebook GPU registra reconstrucción, carga estricta y coincidencia de predicción antes y después de exportar. La verificación externa documentada registra además carga en CPU con weights_only=True e inferencia sintética de (1,3,224,224) a (1,102), con valores finitos. La comprobación sintética no recalcula accuracy sobre Flowers102.

Entorno registrado

GPU Tesla T4: Python 3.13.16, torch 2.14.1+cu130, torchvision 0.29.1+cu130, NumPy 2.4.6, matplotlib 3.10.0, pandas 2.2.3 y scikit-learn 1.6.1. Las versiones se conservan tal como aparecen en el registro de ejecución. La copia limpia del notebook implementa el mismo protocolo; una nueva ejecución puede producir diferencias.

Registro histórico CPU

Se conserva por separado el notebook CPU: Python 3.13.18, torch 2.14.1+cpu y torchvision 0.29.1+cpu. Obtuvo 88.6811% Top-1 y 94.9423% Top-3. Sus figuras permanecen incrustadas y sus cifras se identifican como históricas. El checkpoint y las evidencias externas distribuidas corresponden al registro GPU.

Límites de interpretación

Se conocían resultados históricos de prueba; la selección dentro del registro GPU utilizó validación y la prueba se evaluó después. No se afirma una muestra de prueba nunca examinada, reproducibilidad bit a bit entre dispositivos ni calibración probabilística. Las observaciones visuales describen imágenes y no prueban causas de error. La celda auxiliar de empaquetado no tiene ejecución registrada; entrenamiento, evaluación y exportación sí conservan salidas.

Conclusiones de Arquitectura

La adaptación de fc permitió producir 102 logits con dimensiones verificadas. Descongelar layer4 y fc logró la mayor exactitud de validación entre las seis configuraciones registradas. La auditoría cuantificó el aumento de parámetros entrenables y la evaluación documentó tanto aciertos globales como errores por clase. La exportación conserva pesos, arquitectura, preprocesamiento y resultados para reconstruir el modelo.

Referencias y herramientas

Torchvision: ResNet18 y pesos preentrenados. https://pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html

He et al. Deep Residual Learning for Image Recognition (2016). https://arxiv.org/abs/1512.03385

Oxford Flowers102. https://www.robots.ox.ac.uk/~vgg/data/flowers/102/

Uso de IA: apoyo en organización, redacción y revisión del código. Las cifras se tomaron de los registros y metadatos conservados.